Data Architecture Modernization

Medallion Architecture Implantation

By Pouyan Nikzad

Problem Statement

Platforms can suffer by lack of standardized Medallion Architecture blueprint — tight coupling, limited flexibility, and growing operational risk.

⚡ Performance Bottlenecks

All data must load into SQL; SQL create scalability ceilings and query delays.

🔒 Technology Lock-In

Limited integration capabilities, and lack of Lakehouse platforms. Vendor-bound architecture reduces flexibility.

📊 Governance & Lineage Gaps

Inadequate schema evolution support, missing end-to-end lineage, and no centralized governance across the data lifecycle.

💰 Observability Blind Spots

No proactive cost monitoring, no proactive data quality monitoring, and no efficient, centralized pipelines or data integrity monitoring and anomaly detection across compute, storage, and pipeline layers.

🔁 Redundant Data Layering

Duplicate data storage across different platforms inflates operational costs and pipeline execution times.

Example: loading data into datalake, then sql

🗺️ No Optimization Roadmap

Absence of a standardized design for optimization, storage design and coding leads to inconsistent implementation across teams.

What We're Solving For — Medallion Blueprint Goals

01

Platform Portability

Migrate between architectures (A→B) with minimal disruption. No long-term reliance on any single platform.

02

Open-Source Foundation

Open-source technologies as core engines — high performance, strong governance, portable across environments.

03

End-to-End Lineage

Full, consistent lineage from source to consumption regardless of underlying platform and backward change compatibility — supporting auditability & compliance.

04

Cost Reduction

Eliminate bottlenecks, simplify data flows, and reduce time-to-delivery for data products through eliminating data layers and decoupled compute & storage.

05

Standardized Technical Roadmap

Well-documented blueprint defining how Bronze, Silver, Gold, and Lakehouse components are designed, optimized, built, and evolved.

06

Data Quality & Observability

In-flight validation, SLA monitoring, and consistent quality enforcement across all Medallion layers.

Architecture — Guiding Principles

Platform-Agnostic Design

No dependency on a single platform or architecture using open formats over vendor lock-in and enable long-term adaptability and seamless platform migration.

Example: removing vendor licenses with no operational disruption. Or switching between different architectures.

End-to-End Observability

Full visibility across data quality, cost, lineage, and operational health — from ingestion to consumption by using open-source technologies.

Balance & Evolution

Architecture evolves safely without breaking downstream consumers through layered separation (data maturity) and enforced data contracts and abstraction layers.

Four Pillars of Platform Independence

Open-Source Foundation

› No proprietary license dependency for core data models, formats, or orchestration logic
› Core data, transformation logic, and workflows are defined in open, portable standards
› Licensed platforms and tools act as accelerators—not owners of data or business logic

Code-First Ownership

› All business logic is defined as code and versioned in Git
› Vendor-specific tooling is isolated behind clear interfaces and abstraction layers
› Acceleration tools generate or scaffold code but do not become runtime dependencies

Architectural Abstraction

› Gold layer stable regardless of Silver model
› Macro layer shields consumers from upstream change
› Escape hatch design for future migrations

Efficiency by Design

› Incremental processing and storage optimization reduce unnecessary compute
› Single-copy data access eliminates duplication and › operational overhead
› Precomputed, governed structures reduce query cost and improve performance
› End-to-end observability enables proactive cost control and SLA enforcement

Future State Architecture — Medallion Layers

BRONZE Raw Ingestion	Immutable, append-only storage with full-fidelity preservation · Schema-on-read to absorb upstream change safely · Partitioned for scalable ingestion and replay Tools: Apache Spark . Delta Lake . dbt Core, Databricks
SILVER Enterprise Integration Layer	Enforced schemas and standardized business keys · Data contracts with backward-compatible evolution · Integrated, historical, and auditable datasets · **Abstraction layer** separating source volatility from consumers Tools: dbt Core . Databricks · Apache Spark · Delta Lake
GOLD Consumption Layer	Star schema (Kimball) · Wide OBTs for ML · Pre-aggregated metrics · **Fully decoupled from Silver** Tools: dbt Core . dbt macros . Databricks . Semantic AI · Power BI & notebooks
QUALITY DQ & Observability	End-to-end data quality checks across all layers · Pipeline health, SLA monitoring, and anomaly detection · Cost visibility and proactive alerting Tools: Grafana . dbt test . Apache Spark
GOVERNANCE Metadata & Access Control	Automated column-level lineage from Bronze → Gold · Data classification, tagging, and policy enforcement · Centralized access control and discoverability Tools: Databricks unity catalog . dbt lineage

Platform Capability Matrix — Medallion Architecture

Strategic direction: Databricks as primary compute engine · Fabric for ingestion & BI surface · DV Accelerator for DV2.0 acceleration

Primary / Recommended · Strong support · Supported / Partial · Limited · Not applicable

Capability / Criterion	Layer	DATABRICKS *Primary Compute Engine*	MICROSOFT FABRIC *Ingestion & BI Surface*	dbt + DV Accelerator *dbt DV2.0 Accelerator*
Raw Ingestion & Landing	Bronze	**Strong** Delta Lake on ADLS Gen2; DLT streaming; schema-on-read natively supported	**Primary** Fabric OneLake as primary landing zone; Eventstream for real-time; ADF pipelines for batch	— Not applicable — DV2.0 accelerator operates on Silver layer and above
Batch, CDC & Pipeline Orchestration	Ingestion	**Strong** Databricks Workflows for orchestration; DLT handles CDC via change data feeds natively	**Primary** Fabric Pipelines (ADF-based) for batch; Eventstream for Kafka / CDC / Event Hubs	— No orchestration role — dbt models triggered externally via Workflows or Fabric
Data Vault 2.0 — Hubs, Links, Satellites	Silver	**Strong** Databricks compute runs dbt models; Delta Lake stores DV2 entities at petabyte scale	**Supported** OneLake can host DV tables; limited native DV tooling without external dbt orchestration	**Primary** Core use case — pre-built macros for Hubs, Links, Satellites with hash key generation
PIT / Bridge Tables & Business Vault	Silver	**Strong** Runs dbt models via Databricks SQL warehouse; incremental materialization at scale	**Supported** Supported via custom dbt profile; requires Fabric Lakehouse endpoint configuration	**Primary** Pre-built PIT/Bridge macros and Business Vault soft-rule patterns out of the box
Schema Evolution & Data Contracts	Silver	**Primary** Delta Lake auto schema evolution; Unity Catalog enforces contracts; DLT schema validation	**Supported** Schema-on-read in OneLake; limited native contract enforcement without Unity Catalog	**Supported** dbt schema tests enforce contracts at model level; no storage-layer enforcement
Gold — Star Schema & Wide OBTs	Gold	**Primary** Databricks SQL + dbt Core for Kimball models; Photon engine for fast Gold queries	**Supported** Power BI Direct Lake reads Gold tables; Fabric Warehouse for SQL-based consumption	**Supported** dbt macros accelerate OBT generation; no native Gold-layer tooling beyond dbt helpers
BI, Reporting & ML Feature Store	Consump.	**Primary** Databricks SQL for ad-hoc; MLflow Feature Store; Semantic AI querying; Python notebooks	**Primary** Power BI / Fabric native BI surface; Direct Lake for zero-copy reads from Gold layer	— No consumption-layer role — dbt models feed the layer but DV Accelerator adds no BI capability
Data Quality & Observability	Quality	**Primary** Lakehouse Monitoring (built-in); DLT expectations; Elementary + Grafana on Databricks	**Supported** Fabric DQ rules and pipeline health metrics; limited anomaly detection without add-ons	**Strong** dbt test macros for schema/referential/custom tests; integrates with Elementary for reports
Governance, Lineage & Access Control	Gov.	**Primary** Unity Catalog: column-level lineage, tags, row-level security, data masking, classification	**Strong** Microsoft Purview for lineage & classification; Fabric workspace roles for access control	**Supported** dbt lineage graph provides column-level lineage; no native access control or cataloguing
Performance & Scalability	All	**Primary** Photon engine; serverless SQL; auto-scaling; replaces Synapse SQL pool bottlenecks directly	**Strong** OneLake removes data duplication; Fabric Lakehouse scales independently of Synapse pools	**Supported** Incremental dbt models reduce full-refresh cost; no direct compute or storage optimization
Cost Optimization & Monitoring	Quality	**Primary** Serverless SQL; cluster policies; cost attribution via Unity Catalog tags; budget alerts	**Strong** OneLake storage pricing; Fabric capacity billing; Power BI Premium cost consolidation	**Limited** No cost monitoring — dbt models can be optimized but DV Accelerator adds no cost tooling
CI/CD & Pipeline Automation	Orch.	**Primary** Asset Bundles; git-backed Repos; Workflows API; GitHub Actions native integration	**Strong** Fabric CI/CD via Azure DevOps / GitHub; deployment pipelines for workspace promotion	**Strong** dbt Cloud CI/CD jobs; slim CI for changed models only; GitHub Actions native support

Summary:

Databricks leads across 8/12 criteria — primary engine for Silver, Gold, Quality, Governance, Performance & CI/CD. Replaces Synapse SQL pool bottlenecks.

Fabric is primary for ingestion (OneLake), real-time streaming, and BI (Power BI Direct Lake). Complements Databricks without compute overlap.

DV Accelerator is primary exclusively for DV2.0 Silver layer — Hubs, Links, Satellites, PIT/Bridge macros. No role outside dbt model acceleration.

Architecture Comparison — Current vs Target State

Capability / Criterion	ARCHITECTURE 1 *Synapse SQL Pool*	ARCHITECTURE 2 *Databricks + dbt + DV Accelerator*	Business Impact
Compute Scalability	**Weak** Dedicated SQL pools — fixed DWU capacity, scaling ceilings, query queuing at peak load	**Strong** Photon + serverless auto-scaling; Spark clusters scale from zero to petabytes on demand	*Eliminates query delays and concurrency bottlenecks during peak reporting*
Storage Architecture	**Weak** Data duplicated: ADLS + Synapse SQL pool storage; tight coupling of storage and compute	**Strong** Single source of truth on Delta Lake / OneLake; compute and storage fully decoupled	*Reduces storage costs 30–50% and removes redundant ETL into SQL pools*
Raw Ingestion (Bronze)	**Partial** ADF pipelines land to ADLS, then COPY INTO Synapse — adds extra hop and latency	**Strong** Delta Live Tables + Eventstream land directly to Bronze; schema-on-read, no SQL pool hop	*Simpler pipelines, lower latency, immutable full-fidelity Bronze*
Data Vault 2.0 Execution	**Partial** DV Accelerator generates T-SQL for Synapse; limited by SQL pool concurrency and row store perf	**Strong** DV Accelerator generates dbt models on Databricks; runs on Delta Lake with columnar performance	*DV load windows shrink; vault refreshes parallelise cleanly*
Schema Evolution & Contracts	**Weak** SQL pools require explicit ALTER TABLE; schema drift causes pipeline breaks without warning	**Strong** Delta Lake auto schema evolution; dbt contracts enforced; Unity Catalog validates schemas	*Upstream changes absorbed safely; fewer production incidents*
Gold Modelling (Kimball / OBT)	**Partial** Star schemas in SQL pools; wide OBTs for ML constrained by SQL pool memory and distribution	**Strong** dbt Core on Databricks; Photon-optimised star schema and wide OBTs; no row-count limits	*ML-ready wide tables become economical; ad-hoc analytics unconstrained*
ML & Feature Store	**Missing** No native ML platform; data extracted elsewhere for training, creating duplication and drift	**Native** MLflow + Feature Store native on Databricks; training runs where data lives	*Closes the ML gap completely — no more data copies for model training*
Data Quality & Observability	**Missing** No centralised DQ or anomaly detection; pipeline failures discovered after downstream impact	**Strong** Lakehouse Monitoring + DLT expectations + dbt tests + Elementary dashboards	*Proactive alerting replaces reactive firefighting*
Governance & Column-Level Lineage	**Weak** Purview only; no column-level lineage from Bronze → Gold; fragmented access control	**Strong** Unity Catalog: column-level lineage, row-level security, data masking, centralised tags	*Full auditability and one place to manage access across the data lifecycle*
Cost Transparency	**Weak** DWU billing continuous even when idle; no per-query or per-team cost attribution	**Strong** Serverless SQL pay-per-use; cluster policies; cost tags in Unity Catalog; budget alerts	*Direct cost visibility by team, workload, and criticality*
Real-Time & Streaming	**Weak** Synapse SQL pools are batch-oriented; streaming requires separate Stream Analytics service	**Strong** Structured Streaming + DLT on Databricks; Fabric Eventstream integrated end-to-end	*One platform for batch and real-time — removes architectural fragmentation*
CI/CD & Version Control	**Partial** Synapse workspace + Azure DevOps; limited test coverage	**Strong** Databricks Asset Bundles + Repos + dbt slim CI; GitHub Actions; automated test gating	*Faster, safer releases with tested DV and Gold deployments*
Technology Lock-In	**Weak** Synapse-specific T-SQL, proprietary distribution keys, tight vendor coupling	**Strong** Open Delta Lake format, open-source dbt, Spark SQL — portable and cloud-agnostic	*Future-proof platform — avoids re-architecture cost in 3–5 years*
Operational Risk	**Weak** Growing risk from performance ceilings, observability blind spots, no optimisation roadmap	**Strong** Modern Lakehouse pattern with built-in observability, governance, and scale headroom	*Reduces reliability risk; aligns with industry-standard Medallion patterns*

Verdict: Architecture 1 (Synapse + Legacy Zones) is weak or missing on 9 of 14 criteria — performance ceilings, no ML, no observability, no column-level lineage, and vendor lock-in.

Architecture 2 (Databricks + dbt + DV Accelerator) is strong or native on 13 of 14 — closing every gap while preserving *DV Accelerator* DV2.0 investment.

dbt Layer Comparison — With vs Without dbt Orchestration

Capability / Criterion	ARCHITECTURE A — WITHOUT dbt *Databricks + DV Accelerator*	ARCHITECTURE B — WITH dbt *Databricks + dbt Core + DV Accelerator*	Business Impact
Transformation Framework	**Partial** No standard method to build Gold transformations, hand-coded as notebooks or SQL scripts	**Strong** dbt Core as unified framework from Silver to Gold; DV tools models run inside dbt project	*Single transformation language across all layers — no mixed notebooks/SQL scripts*
Gold Layer Modelling	**Weak** No standardised Gold layer tooling; teams build Kimball/OBTs inconsistently	**Strong** dbt models with ref() and source() macros; standardised Kimball, OBT, and metrics layer	*Consistent Gold layer patterns across teams; removes custom notebook sprawl*
Data Quality Testing	**Weak** No native framework for Silver or Gold-layer on business-rule tests	**Strong** dbt test macros: unique, not_null, relationships, custom SQL tests on every model	*Testable Gold layer — DQ coverage extends beyond DV2.0 into business-facing models*
Documentation & Data Dictionary	**Weak** Gold or Silver layer documentation is manual or missing	**Strong** dbt docs auto-generates end-to-end catalogue with descriptions, tests, and lineage visualization	*Self-service documentation for business users; reduces data team support burden*
Column-Level Lineage	**Partial** Unity Catalog for storage; no lineage for Gold transformations	**Strong** dbt lineage graph end-to-end (Silver → Gold); complements Unity Catalog column-level tracking	*Full lineage closes the Bronze → Gold visibility gap called out in problem statement*
Version Control & Git Workflow	**Partial** DV tool exports to git; Gold models in notebooks, sql often live outside version control	**Strong** Every model, test, and macro is a .sql file in git; pull request reviews, branch deployments	*All transformations version-controlled; proper code review for Gold models*
CI/CD & Automated Testing	**Weak** no automated testing for Silver and Gold models on PR	**Strong** dbt slim CI runs only changed models; GitHub Actions gates merges on test pass	*Faster, safer releases; failed tests block bad code from reaching production*
Incremental Model Strategy	**Partial** DV tool handles DV incrementals; Gold-layer incremental logic must be hand-coded per model	**Strong** dbt incremental materialisations with merge, append, and insert_overwrite strategies built-in	*Reduces Gold-layer compute cost 60–80% vs full refresh; less custom logic to maintain*
Environment Management	**Partial** Dev/prod workspaces in Databricks; no systematic env isolation for transformations	**Strong** dbt targets (dev/staging/prod); every developer runs models in isolated schema/catalog	*Developers never collide; safer experimentation without prod risk*
Modularity & Reusability	**Weak** No macro system beyond DV tool macros; Gold-layer logic duplicated across notebooks and sql	**Strong** Custom Jinja macros reusable across models; dbt packages (dbt_utils, dbt_expectations) extend capability	*Less code duplication; shared business logic maintained in one place*
Metrics Layer (Semantic)	**Missing** No metrics layer; BI teams redefine KPIs in Power BI, creating inconsistent numbers across reports	**Native** dbt Semantic Layer / Metrics: single source of truth for revenue, margin, KPIs queried by BI tools	*Eliminates 'which number is right?' — one metric definition, one answer everywhere*
Developer Onboarding	**Partial** DV accelerator helps for DV; Gold developers learn notebook patterns, sql and ad-hoc, no standard	**Strong** Industry-standard dbt skill; rich community, tutorials, open-source ecosystem	*Hire dbt-skilled engineers off the market; smaller ramp-up curve*
Cost: Tooling Licensing	**Strong** Only DV tool license required; no additional transformation tool cost	**Strong** dbt Core is open-source (free)	*Adding dbt costs zero in licensing; returns compound via productivity gains*
Vendor & Tool Lock-In	**Partial** Gold logic locked into Databricks notebooks; hard to migrate or test outside platform	**Strong** dbt models are plain SQL files; portable across Databricks, Snowflake, BigQuery, Postgres	*Future-proof transformation layer — not tied to a single compute vendor*

Verdict: Without dbt, DV tool covers only the Silver DV layer — Gold, testing, docs, CI/CD, and metrics become ad-hoc work that doesn't scale.

dbt Core adds zero licensing cost but provides the framework that turns Databricks + DV Accelerator into a disciplined, industry-standard Medallion platform.

Observability Comparison — Why Grafana Completes the Stack

Capability / Criterion	ARCHITECTURE A — WITHOUT Grafana *Databricks + dbt*	ARCHITECTURE B — WITH Grafana *Databricks + dbt + Grafana*	Business Impact
Pipeline Health Dashboards	**Partial** Databricks Workflows UI shows job runs; dbt docs shows test results — but no unified pipeline	**Strong** Grafana panels unify Workflows + dbt + Delta + Spark metrics in one operational dashboard	*Single pane of glass — operators see the entire pipeline state at a glance*
Real-Time Monitoring	**Weak** Databricks job UI refreshes manually; no live dashboards for streaming jobs or active workloads	**Strong** Live-refreshing Grafana dashboards with sub-minute granularity for streaming and batch jobs	*Live operational visibility for on-call teams during incidents*
Proactive Alerting	**Weak** Databricks job-failure email; dbt test failures appear in logs — no proactive threshold alerts	**Strong** Grafana Alerting: thresholds, anomaly detection, multi-channel routing (Slack, PagerDuty, Teams, email)	*Issues caught before downstream impact — replaces reactive firefighting*
SLA & SLO Tracking	**Missing** No SLA tracking; pipeline freshness measured manually or via custom queries on every check	**Strong** Grafana SLO panels for freshness, completion time, error rate; trend over weeks/months	*Quantifiable platform reliability — exec-ready uptime and freshness reporting*
Cost Monitoring	**Partial** Databricks system tables expose DBU usage; cost analysis requires separate notebook or BI tool	**Strong** Grafana dashboards on system.billing tables; per-team, per-workload, per-job cost trends with alerts	*Daily cost visibility prevents bill shock; budget alerts before overrun*
Anomaly Detection	**Weak** dbt tests catch data anomalies but only on test runs; no time-series anomaly detection on metrics	**Strong** Grafana ML alerts detect deviation in row counts, latency, freshness, and DQ scores over time	*Detects silent degradation that point-in-time tests miss*
Cross-System Correlation	**Missing** Cannot correlate Databricks events with Fabric ingestion, Kafka lag, or downstream Power BI	**Strong** Multi-data-source dashboards: Databricks + Fabric + eventhub + cloud metrics correlated on one timeline	*Faster root cause analysis when problems span multiple systems*
Historical Trend Analysis	**Weak** Databricks UI shows recent runs only; long-term trends require manual export and analysis	**Strong** Months/years of metric history in Grafana with retention policies; trend lines built-in	*Capacity planning grounded in actual long-term workload trends*
Custom Metrics & KPIs	**Partial** Custom metrics require building bespoke notebooks or jobs; no standard visualization layer	**Strong** Any SQL or Prometheus metric becomes a Grafana panel; reusable across dashboards	*Teams build their own dashboards without engineering bottleneck*
On-Call & Incident Response	**Weak** Engineers ssh into Databricks UI; no runbooks linked to alerts; manual triage every time	**Strong** Grafana annotations + linked runbooks; alert context (graphs, logs, dashboards) at incident time	*Faster MTTR; junior engineers can handle incidents using documented runbooks*
Stakeholder Reporting	**Weak** Status updates compiled manually from Databricks logs and dbt run reports — slow and inconsistent	**Strong** Public Grafana dashboards or scheduled PDF/Slack reports for execs, data stewards, business teams	*Self-service status visibility — fewer 'is the data fresh?' messages to data team*
Vendor Independence	**Partial** Tied to Databricks-native UIs and dbt's own UI; observability locked to each tool's surface	**Strong** Grafana is open-source, vendor-agnostic; same dashboards work if Databricks or dbt are replaced later	*Future-proof observability layer — survives platform changes*
Cost: Tooling Licensing	**Strong** No additional tooling cost — Databricks and dbt Core only	**Strong** Grafana OSS is free and self-hosted; Grafana Cloud has free tier; minimal incremental cost	*Adding Grafana costs near zero; ROI from prevented incidents pays back quickly*
Operational Maturity	**Weak** Reactive operations: problems found by users; SLAs informal; no metrics-driven improvement loop	**Strong** Proactive ops: alerts before users notice; metrics-driven reliability culture; data SRE practices	*Platform graduates from 'works most of the time' to enterprise-grade reliability*

Verdict: Databricks + dbt monitor data quality and runs but lack a unified operational dashboard, alerting, and SLA tracking — leaving observability blind spots.

Grafana adds the missing observability layer at near-zero cost — proactive alerts, real-time dashboards, and SLA tracking that turn ops from reactive to proactive.

Project Roadmap — High-Level Phases

Three sequential phases + cross-cutting workstreams (security, DevOps, change management, stakeholder comms)

Phase 1 of Pilot

Bronze / Ingestion

Foundations & Proof of Value

12 weeks

Prerequisite: Discovery, scope & Platform foundation

Candidate sources; define success criteria & KPIs vs current state, Provision Databricks + Unity Catalog + dbt

Bronze build

Delta Lake landing with schema-on-read; partitioning strategy; batch + real-time ingestion patterns

DevOps & standards

Git repos, naming conventions and dbt project scaffolding

Proof of value

Architecture achievements, cost benchmarks vs Synapse, detailed technical roadmap for rollout

Phase 2

Silver / Governance / DQ / Observability

Core Integration & Trust Layer

TBD

Silver DV2.0 build

Hubs, Links, Satellites, Business Vault, PIT / Bridge tables via DV Accelerator + dbt on Databricks

Historical backfill

Load history from source systems into the vault; reconciliation against Synapse for parity

Data Quality framework

dbt tests (schema, referential, custom); DLT expectations; Elementary + Grafana observability

Governance rollout

Unity Catalog tags & classification; column-level lineage Bronze → Silver

Abstraction & validation

Macro layer decouples DV from Gold

Phase 3

Gold / Consumption

Business Value & Cutover

TBD

Gold modelling

Kimball star schemas, wide OBTs for ML, metrics layer built from Silver abstraction (tool-agnostic)

Consumption integration

Power BI Direct Lake, ML Feature Store, Databricks SQL, notebooks, API endpoints

UAT & benchmarking

Business user acceptance testing; performance vs Synapse baseline

Parallel run & cutover

Dual-run new vs old platform; data reconciliation; report re-pointing; Synapse decommission plan

End-to-end assurance

Full lineage Bronze → Gold; schema contract validation; Semantic AI (Semantic AI) integration; hypercare

Across all phases →

- Change Management & Training
- Security & Compliance
- Cost & Capacity Governance
- Stakeholder Communication
- Risk & Issue Tracking

Deep Dive and Examples

Example of Abstraction Layer:

With abstraction layer:

GOLD

- sales_fact.sql — calls get_customer()
- customer_dim.sql — calls get_customer()
- revenue_obt.sql — calls get_customer()

MACRO

- get_customer() macro — the stable interface — gold calls this, nothing else

macro resolves to →

SILVER

- hub_customer — Data Vault hub
- sat_customer_detail — Data Vault satellite
- link_order_customer — Data Vault link

Gold SQL now looks like:

```
SELECT customer_id, name, email
FROM {{ get_customer() }}
```

Gold has **no idea** what is behind that macro. It just knows it gets a clean table with customer_id, name, email. Data Vault, Kimball, flat table — it doesn't matter.

Without abstraction layer:

GOLD

- sales_fact.sql — calls DV tables directly
- customer_dim.sql — calls DV tables directly
- revenue_obt.sql — calls DV tables directly

SILVER

- hub_customer — Data Vault hub
- sat_customer_detail — Data Vault satellite
- link_order_customer — Data Vault link

Every gold model contains SQL like:

```
SELECT h.customer_hk, s.name, s.email
FROM hub_customer h
JOIN sat_customer_detail s ON h.customer_hk = s.customer_hk
```

The gold model **knows it is talking to Data Vault**. Hub names, satellite names, hash key columns — all hardcoded inside gold SQL.

If you decide to abandon Data Vault later:

hub_customer.sql	rewrite
sat_customer_detail.sql	rewrite
link_order_customer.sql	rewrite
sales_fact.sql	rewrite
customer_dim.sql	rewrite
revenue_obt.sql	rewrite
...every other gold model	rewrite

Changing silver forces you to rewrite gold too — and gold is what your BI reports and ML pipelines depend on. The blast radius is the entire platform.

Escape Hatch Design

Compute Layer (Databricks vs Fabric vs Spark Anywhere)

```
Business Logic
  ↓
Spark Execution Interface
  ↓
Databricks | Fabric | EMR
```

Orchestration (Airflow vs ADF vs Platform Schedulers)
Pipelines defined as logical workflows, not vendor DAGs.
Ingest → Validate → Transform → Publish

Storage Layer (Delta / Iceberg / Parquet)
Data access standardized through open table formats

Same table usable across:

- Databricks
- Fabric
- Open-source Spark

Transformations (dbt as Contract Boundary)

```
dbt Model
  ↓
Warehouse Adapter
  ↓
Any SQL Engine
```

Example of Data Contracts:

Contract for a source table (raw → bronze) using sources + tests + freshness

```yaml
version: 2

sources:
  - name: crm
    schema: raw_crm
    tables:
      - name: customers
        description: "Raw customers from CRM (ingested as-is)."
        freshness:
          warn_after: {count: 2, period: hour}
          error_after: {count: 6, period: hour}
        loaded_at_field: _ingested_at

        columns:
          - name: customer_id
            description: "Natural key from CRM."
            tests:
              - not_null
              - unique

          - name: status
            description: "Customer lifecycle state."
            tests:
              - not_null
              - accepted_values:
                  values: ["ACTIVE", "INACTIVE", "PROSPECT"]

          - name: email
            description: "Email address (may be null if unknown)."
            tests:
              - dbt_utils.expression_is_true:
                  expression: "email is null or email like '%@%'"
```

Contract for a model schema using contract: enforced (dbt “model contracts”)

```yaml
version: 2

models:
  - name: dim_customer
    description: "Conformed customer dimension used by analytics and ML features."
    config:
      materialized: table
      contract:
        enforced: true

    columns:
      - name: customer_sk
        data_type: bigint
        description: "Surrogate key."
        tests:
          - not_null
          - unique

      - name: customer_id
        data_type: varchar
        description: "Natural key from CRM."
        tests:
          - not_null

      - name: status
        data_type: varchar
        description: "Normalized status."
        tests:
          - not_null
          - accepted_values:
              values: ["ACTIVE", "INACTIVE", "PROSPECT"]

      - name: effective_start_ts
        data_type: timestamp
        description: "SCD2 start timestamp."
        tests: [not_null]

      - name: effective_end_ts
        data_type: timestamp
        description: "SCD2 end timestamp (null means current)."
```

Referential integrity contract (fact ↔ dimension) with relationships

```yaml
version: 2

models:
  - name: fct_orders
    description: "Order facts."
    columns:
      - name: order_id
        tests: [not_null, unique]

      - name: customer_sk
        description: "FK to dim_customer."
        tests:
          - not_null
          - relationships:
              to: ref('dim_customer')
              field: customer_sk
```

Business-rule contract with custom checks (e.g., “amount >= 0”)

```yaml
columns:
  - name: order_amount
    description: "Total order amount in CAD."
    tests:
      - not_null
      - dbt_utils.expression_is_true:
          expression: "order_amount >= 0"
```

Example of Grafana Monitoring and Alerting Logic:

Data Pipeline Monitoring (Batch + Streaming)

- Job: bronze_ingestion_stream
- Micro-batch interval: 5 min
- SLA breach alert if lag > 15 min

Cost Monitoring & Resource Usage

Trigger alert if:

- Cost increases > 30% day-over-day
- Cluster idle > 40% of runtime

Data Quality Monitoring

Model: fct_orders

- Freshness SLA: < 2 hours
- Failing tests: 3
- Affected dashboards: 5

Conditional Monitoring

IF

- Consumer lag > 1M events

FOR

- 10 minutes

THEN

- Critical alert to on-call

dbt Test Failures Trend or Data Contract Breaks / Schema Drift

IF

- Failed dbt tests (severity=error) > 0
- AND model_tier = "tier1"

FOR

- 5 minutes

THEN

- Critical alert to on-call
- Include: failing_test_names, affected_models, last_successful_run_time

IF

- schema_breaking_changes > 0
  - Examples of breaking change types:
    - column_dropped
    - column_renamed
    - type_changed_incompatible

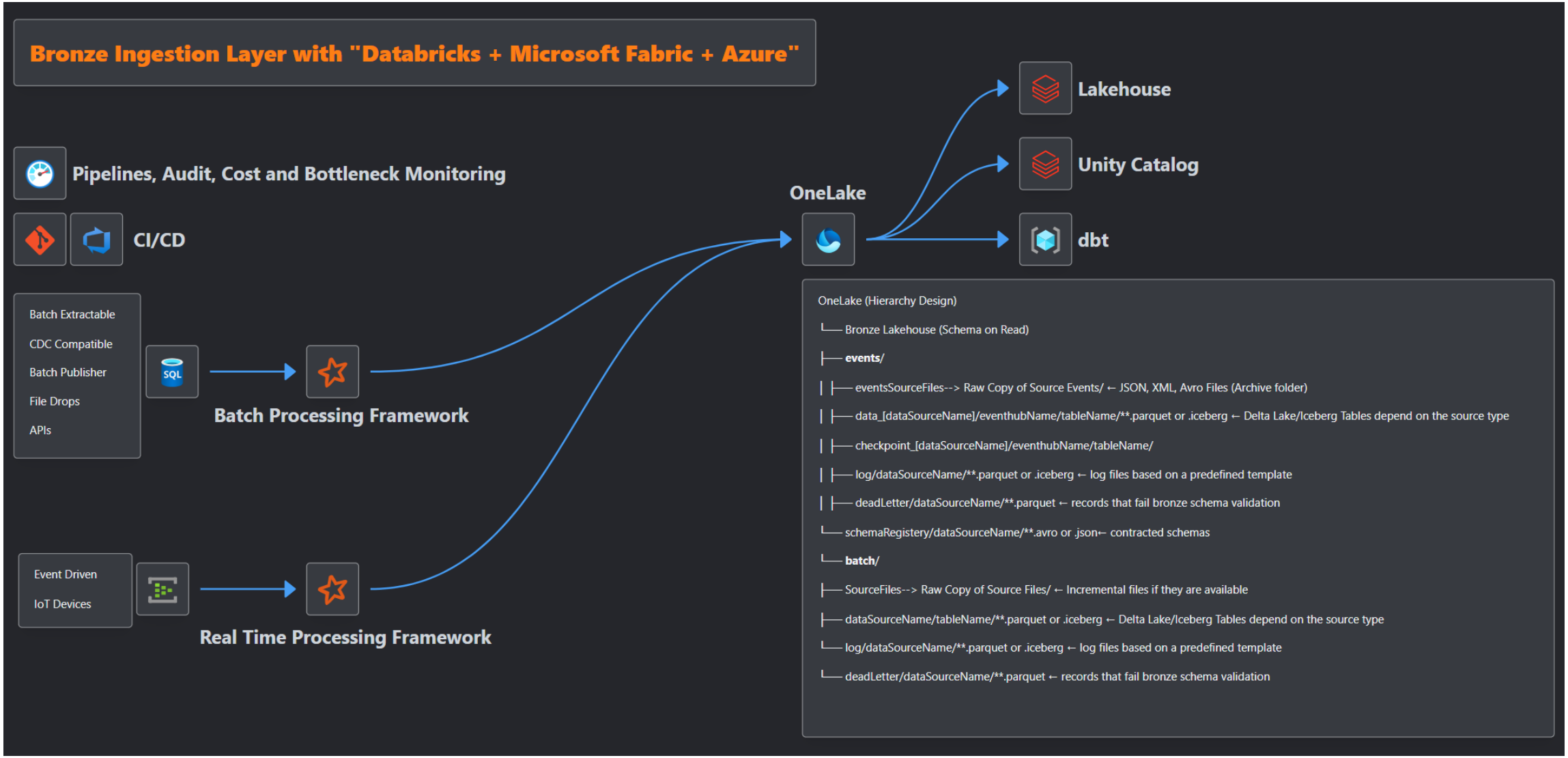
FOR

- 1 minute

THEN

- Critical alert to on-call
- Include: source/topic, old_schema_version, new_schema_version, change_summary

Bronze Layer — Ingestion Framework, Batch and Real-Time Storage Design



Bronze Layer — Ingestion Framework, Batch and Real-Time Orchestration Design

Batch Processing Framework

- Batch Extractable
- CDC Compatible
- Batch Publisher
- File Drops
- APIs

Batch Processing Framework

Microsoft Fabric (Main Portal)

- **Orchestration Tools: ADF/Apache Airflow**
- Trigger Spark Job, standard parametrize Python functions (File/JDBC)
- Trigger Databricks Auto Loader batch
- Trigger Databricks DLT pipeline
- **Monitoring Tools:**
- Log monitoring with threshold and alerts for task duration, source lag and runtime errors
- Workflow failure and alerts in Fabric Monitor hub and ADF
- Fabric and Databricks cluster tags, ADF DIU cost and daily threshold

OneLake Query and Optimization

Databricks, Lakehouse:

- Endpoint: **Databricks SQL Warehouse (zero copy data)**
- Optimization: Liquid clustering + VACUUM + Partitioning
- Governance: Unity Catalog (Lineage, PII, Schema, Access, Breaking Change Prevention)

DBT:

- Endpoint: DBT UI
- Version control over tables, schema management, table definitions, DBT test process, whole platform lineage (not just UC Databricks)

Vault Speed:

- Endpoint: Vault Speed UI
- Raw vault

DATA ARCHITECTURE MODERNIZATION

Thank You

Questions, discussion & next steps

Appendix: Databricks + dbt + Grafana

Core benefits of dbt

1. dbt Enables Multiple Architectures in Parallel: Data Vault 2.0 (Silver), Star Schema / Dimensional (Gold BI), One-Big-Table (Gold ML), Streaming → Batch convergence, Domain-oriented / Data Mesh outputs
2. Transformation as Code (Analytics Engineering)
    - SQL-based transformations versioned in Git
    - Clear separation between ingestion and transformation
3. Modularity & Reusability
    - Encourages DRY logic via refs, macros, and Jinja templating
    - Easier refactoring and incremental evolution
4. Built-in Data Quality & Contracts
    - Native tests (not null, unique, accepted values, relationships)
    - Schema tests act as layer contracts (Bronze → Silver → Gold)
    - Fail fast before bad data propagates downstream
5. Dependency Management & Lineage
    - Clear upstream/downstream impact analysis
    - Strong integration with cataloging & lineage tools
6. Incremental & Cost-Efficient Processing
    - Built-in incremental models (merge, insert_overwrite)
    - Minimal recomputation → lower compute cost
    - Ideal for large lakehouse tables (Delta/Iceberg)
7. Observability & Documentation
    - Auto-generated documentation site
    - Column-level descriptions live with the code
    - Improves trust, discoverability, and onboarding
8. Platform Independence
    - Same dbt project can run on Databricks, Fabric, Snowflake, Synapse
    - Reduces vendor lock-in risk
9. Team Enablement & Governance
    - Empowers analytics engineers and data modelers
    - Consistent patterns enforced across teams
    - Shifts transformation ownership closer to the business

Core Benefits of Grafana

- Real-time observability
- Near-instant visibility into system health, jobs, streams, and infrastructure (seconds, not minutes).
- Single pane of glass
- Monitor Databricks, Fabric, Kafka/Event Hubs, Airflow, databases, cloud infra, and apps in one place.
- Proactive alerting
- Threshold, anomaly, and SLO-based alerts with routing to Teams, Slack, PagerDuty, email.
- Cost & FinOps monitoring
- Track compute usage, cost trends, and spikes before invoices arrive.
- Operational metrics visibility
- Deep insight into runtime signals (latency, throughput, lag, retries, memory, failures) — not just query results.
- Platform reliability (SRE-style)Enables SLAs, SLOs, error budgets, uptime tracking, and reliability reporting.
- Scales across teams and environments
- Works consistently across dev, test, perf, and prod.
- Tool-agnostic & vendor-neutral
- Open, extensible, integrates with almost any data source or metrics backend.
- Correlation & root-cause analysis
- Correlate failures, performance drops, and cost spikes across systems.
- Shared operational visibility
- Engineers, platform teams, and leaders see the same health signals.